MergeSort

Merge Sort Algorithms

Goal

Input: An array X of size $n = 2^k$ with $k \in \mathbb{N}$.
Output: The same array X, sorted in nondecreasing order, i.e., $X[i] \leq X[j] \quad \forall 1 \leq i < j \leq n$.

Example

Input array

$X =$

12	-5	-7	51	6	28	3	-8
1	2	3	4	5	6	7	8

Sorted array

$X =$

-8	-7	-5	3	6	12	28	51
1	2	3	4	5	6	7	8

Divide-and-Conquer Approach

Main idea

Simple parallel MergeSort follows a divide-and-conquer strategy:

- split the input array X into two halves X_1 and X_2

Divide-and-Conquer Approach

Main idea

Simple parallel MergeSort follows a divide-and-conquer strategy:

- split the input array X into two halves X_1 and X_2
- sort the two halves recursively

Divide-and-Conquer Approach

Main idea

Simple parallel MergeSort follows a divide-and-conquer strategy:

- split the input array X into two halves X_1 and X_2
- sort the two halves recursively
- merge the two sorted halves X_1 and X_2 back into X

Divide-and-Conquer Approach

Main idea

Simple parallel MergeSort follows a divide-and-conquer strategy:

- split the input array X into two halves X_1 and X_2
- sort the two halves recursively
- merge the two sorted halves X_1 and X_2 back into X

Observation. The global structure of the algorithm is very simple:

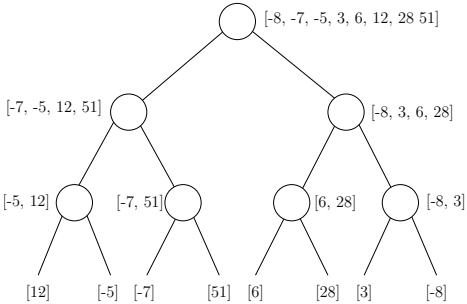
- the recursion naturally defines subproblems
- independent subproblems can be processed in **parallel**
- the key operation is the *merge* used to combine the two halves

Tree Representation

Balanced binary tree T

The recursive structure can be represented by a balanced binary tree T:

- each leaf corresponds to one input element

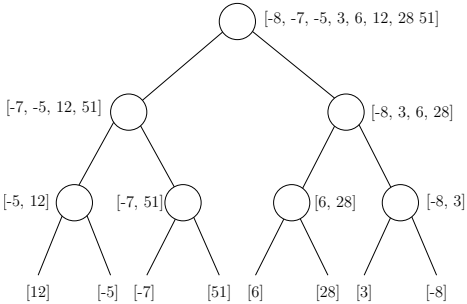


Tree Representation

Balanced binary tree T

The recursive structure can be represented by a balanced binary tree T:

- each leaf corresponds to one input element
- each internal node represents the merge of its two children

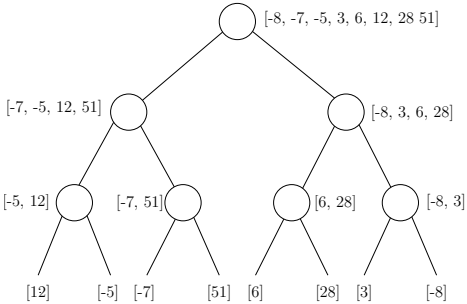


Tree Representation

Balanced binary tree T

The recursive structure can be represented by a balanced binary tree T:

- each leaf corresponds to one input element
- each internal node represents the merge of its two children
- the root stores the final sorted sequence



Tree Representation

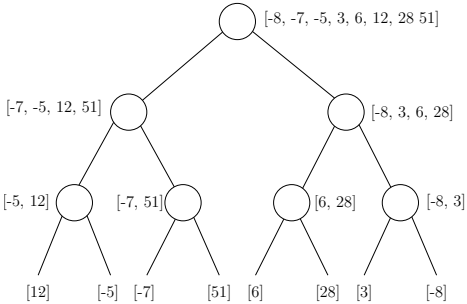
Balanced binary tree T

The recursive structure can be represented by a balanced binary tree T:

- each leaf corresponds to one input element
- each internal node represents the merge of its two children
- the root stores the final sorted sequence

Why is this useful?

- the height of T is $\log n$
- each level contains independent merges
- the total cost is obtained from #levels $\times$ cost per level



Pseudocode

ALGORITHM *Simple_Mergesort(X):*
begin

for $1 \leq j \leq n$ **pardo**
    Set $L[0, j] := X[j]$
for $h = 1$ **to** $\log n$ **do**
    for $1 \leq j \leq n/2^h$ **pardo**
        $L[h, j] \leftarrow merge(L[h-1, 2j-1], L[h-1, 2j])$
end

Notation. The index h denotes the level of the tree, and j the position of the node from left to right within that level. Hence, $L[h, j]$ is the sorted list associated with that node, namely the sorted version of the block $X[(j-1)2^h + 1 \ldots j2^h]$.

Pseudocode

ALGORITHM *Simple_Mergesort(X):*
begin

```
for 1 ≤ j ≤ n pardo
    Set L[0, j] := X[j]
for h = 1 to log n do
    for 1 ≤ j ≤ n/2^h pardo
        L[h, j] ← merge(L[h−1, 2j−1], L[h−1, 2j])
end
```

Observations

- leaves contain single elements

Notation. The index h denotes the level of the tree, and j the position of the node from left to right within that level. Hence, $L[h, j]$ is the sorted list associated with that node, namely the sorted version of the block $X[(j-1)2^h + 1 \ldots j2^h]$.

Pseudocode

ALGORITHM *Simple_Mergesort(X):*
begin

for $1 \leq j \leq n$ **pardo**
    Set $L[0, j] := X[j]$
for $h = 1$ **to** $\log n$ **do**
    for $1 \leq j \leq n/2^h$ **pardo**
        $L[h, j] \leftarrow merge(L[h-1, 2j-1], L[h-1, 2j])$
end

Observations

- leaves contain single elements
- each level performs **independent** merges

Notation. The index h denotes the level of the tree, and j the position of the node from left to right within that level. Hence, $L[h, j]$ is the sorted list associated with that node, namely the sorted version of the block $X[(j-1)2^h + 1 \ldots j2^h]$.

Pseudocode

ALGORITHM *Simple_Mergesort(X):*
begin

for $1 \leq j \leq n$ **pardo**
    Set $L[0, j] := X[j]$
for $h = 1$ **to** $\log n$ **do**
    for $1 \leq j \leq n/2^h$ **pardo**
        $L[h, j] \leftarrow merge(L[h-1, 2j-1], L[h-1, 2j])$
end

Observations

- leaves contain single elements
- each level performs **independent** merges
- the root contains the entire sorted sequence

Notation. The index h denotes the level of the tree, and j the position of the node from left to right within that level. Hence, $L[h, j]$ is the sorted list associated with that node, namely the sorted version of the block $X[(j-1)2^h + 1 \ldots j2^h]$.

Pseudocode

ALGORITHM *Simple_Mergesort(X):*
begin

for $1 \leq j \leq n$ **pardo**
    Set $L[0, j] := X[j]$
for $h = 1$ **to** $\log n$ **do**
    for $1 \leq j \leq n/2^h$ **pardo**
        $L[h, j] \leftarrow merge(L[h-1, 2j-1], L[h-1, 2j])$
end

Observations

- leaves contain single elements
- each level performs **independent** merges
- the root contains the entire sorted sequence
- tree-based structure, read bottom-up, left-to-right

Notation. The index h denotes the level of the tree, and j the position of the node from left to right within that level. Hence, $L[h, j]$ is the sorted list associated with that node, namely the sorted version of the block $X[(j-1)2^h + 1 \ldots j2^h]$.

How to read the pseudocode

Level $h = 0$

Each processor assigns one input element to one leaf:

$$L[0, j] = X[j], \qquad 1 \leq j \leq n.$$

Generic level $h \geq 1$

For each internal node at level h, we merge the pair of lists stored in its two children:

$$L[h, j] = merge(L[h-1, 2j-1], L[h-1, 2j]).$$

How to read the pseudocode

Level $h = 0$

Each processor assigns one input element to one leaf:

$$L[0, j] = X[j], \qquad 1 \leq j \leq n.$$

Generic level $h \geq 1$

For each internal node at level h, we merge the pair of lists stored in its two children:

$$L[h, j] = merge(L[h-1, 2j-1], L[h-1, 2j]).$$

Questions:

- Why can the inner `for` be `pardo`?

How to read the pseudocode

Level $h = 0$

Each processor assigns one input element to one leaf:

$$L[0, j] = X[j], \qquad 1 \leq j \leq n.$$

Generic level $h \geq 1$

For each internal node at level h, we merge the pair of lists stored in its two children:

$$L[h, j] = merge(L[h-1, 2j-1], L[h-1, 2j]).$$

Questions:

- Why can the inner `for` be `pardo`?
  Because each iteration handles a distinct merge and writes to a distinct output list, with no interference among iterations.

How to read the pseudocode

Level $h = 0$

Each processor assigns one input element to one leaf:

$$L[0, j] = X[j], \qquad 1 \leq j \leq n.$$

Generic level $h \geq 1$

For each internal node at level h, we merge the pair of lists stored in its two children:

$$L[h, j] = merge(L[h-1, 2j-1], L[h-1, 2j]).$$

Questions:

- Why can the inner `for` be `pardo`?
  Because each iteration handles a distinct merge and writes to a distinct output list, with no interference among iterations.
- Why can the outer `for` not be `pardo`?

How to read the pseudocode

Level $h = 0$

Each processor assigns one input element to one leaf:

$$L[0, j] = X[j], \qquad 1 \leq j \leq n.$$

Generic level $h \geq 1$

For each internal node at level h, we merge the pair of lists stored in its two children:

$$L[h, j] = merge(L[h-1, 2j-1], L[h-1, 2j]).$$

Questions:

- Why can the inner `for` be `pardo`?
  Because each iteration handles a distinct merge and writes to a distinct output list, with no interference among iterations.
- Why can the outer `for` not be `pardo`?
  Because level h requires the values computed at level $h-1$, so the iterations over h are not independent and must be executed sequentially.

How to read the pseudocode

Level $h = 0$

Each processor assigns one input element to one leaf:

$$L[0, j] = X[j], \qquad 1 \leq j \leq n.$$

Generic level $h \geq 1$

For each internal node at level h, we merge the pair of lists stored in its two children:

$$L[h, j] = merge(L[h-1, 2j-1], L[h-1, 2j]).$$

Questions:

- Why can the inner `for` be `pardo`?
  Because each iteration handles a distinct merge and writes to a distinct output list, with no interference among iterations.
- Why can the outer `for` not be `pardo`?
  Because level h requires the values computed at level $h-1$, so the iterations over h are not independent and must be executed sequentially.

Fundamental invariant

At the end of iteration h, each node (h, j) contains the elements of its subtree in non-decreasing order.

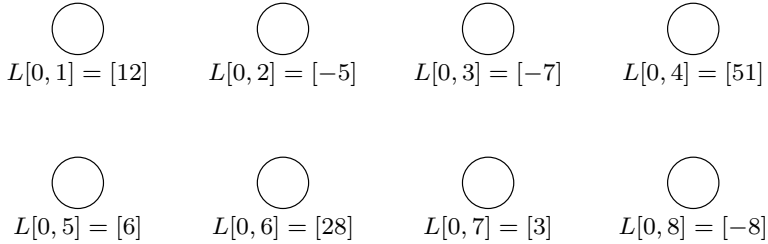
Example - Initialization

Initialization

At level $h = 0$, each input element is assigned to one leaf: $L[0, j] = [X[j]]$ for $1 \leq j \leq 8$.

At level $h = 0$

- $L[0, 1] = [12]$
- $L[0, 2] = [-5]$
- $L[0, 3] = [-7]$
- $L[0, 4] = [51]$
- $L[0, 5] = [6]$
- $L[0, 6] = [28]$
- $L[0, 7] = [3]$
- $L[0, 8] = [-8]$



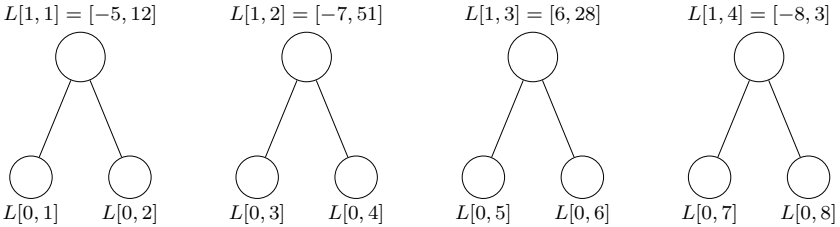
Example 4.4 — first merge level on 8 elements

Input

$X = [12, -5, -7, 51, 6, 28, 3, -8]$

During iteration $h = 1$

- $L[1,1] = [-5, 12]$
- $L[1,2] = [-7, 51]$
- $L[1,3] = [6, 28]$
- $L[1,4] = [-8, 3]$



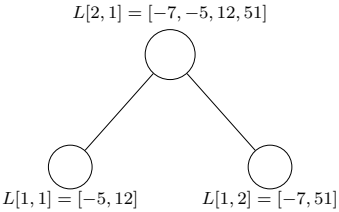
Example 4.4 — second merge level on 8 elements

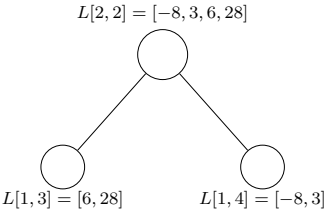
Input

$X = [12, -5, -7, 51, 6, 28, 3, -8]$

During iteration $h = 2$

- $L[2,1] = [-7, -5, 12, 51]$
- $L[2,2] = [-8, 3, 6, 28]$





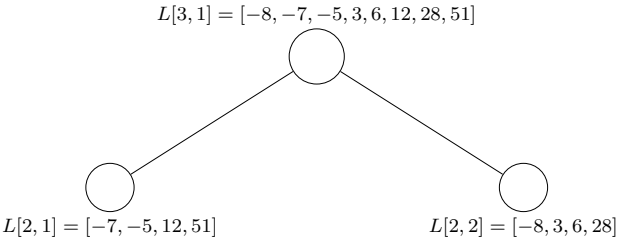
Example 4.4 — final merge level on 8 elements

Input

$X = [12, -5, -7, 51, 6, 28, 3, -8]$

During iteration $h = 3$

- $L[3, 1] = [-8, -7, -5, 3, 6, 12, 28, 51]$



Proof sketch

Theorem 1

For every node v of the balanced tree T, algorithm Simple_Mergesort generates the sorted list $L[v]$ of the elements contained in the subtree rooted at v. The running time is $O(\log^2 n)$ and the total number of operations is $O(n \log n)$.

Correctness

The correctness follows from the next lemma: for every level h and every index j, the list $L[h, j]$ is the sorted version of the block

$$X[(j-1)2^h + 1 \dots j2^h].$$

Applying the lemma to the root gives the desired conclusion.

Cost

- there are $\log n$ levels
- at each level the total number of involved elements is n
- the *merge* procedure seen has time complexity $T(n) = O(\log n)$ and work $W(n) = O(n)$, on a CREW PRAM model

Correctness lemma for *Simple_Mergesort*

Lemma 2

For every level h and every index j with $1 \leq j \leq n/2^h$, the list $L[h, j]$ is the sorted version of the block

$$X[(j-1)2^h + 1 \ldots j2^h].$$

Proof.

We prove the claim by induction on the level h.

- **Base case ($h = 0$).** For every j, the algorithm sets $L[0, j] = X[j]$. Hence $L[0, j]$ is a sorted list of length 1, namely the sorted version of the block $X[j \ldots j]$.
- **Inductive step.** Assume the claim true for level $h - 1$. Consider a node (h, j), whose left child is $(h-1, 2j-1)$ and whose right child is $(h-1, 2j)$. By the inductive hypothesis, $L[h-1, 2j-1]$ and $L[h-1, 2j]$ are the sorted versions of the two consecutive subblocks associated with these children; Since *merge* correctly merges two sorted lists, the list

  $$L[h, j] = merge(L[h-1, 2j-1], L[h-1, 2j])$$

  is the sorted version of the whole block $X[(j-1)2^h + 1 \ldots j2^h]$.

$\square$

Operational lemma for the analysis

Lemma 3

In each iteration of the outer loop, the total number of elements involved in all merges is exactly n.

Proof.

At level h, the n input elements are partitioned into blocks of size 2^h. Therefore, the number of blocks at that level is

$$\frac{n}{2^h}.$$

Each block is obtained by merging two lists of length 2^{h-1}, so each merge involves exactly 2^h elements. Hence, the total number of elements involved in all merges at level h is

$$\frac{n}{2^h} \cdot 2^h = n.$$

Consequently, if each merge performs linear work in its input size, then the total work of one level is $O(n)$. □

Conclusion of the proof of Theorem 1

Proof of Theorem 1.

- By Lemma 2, applied to the root $(\log n, 1)$, the list $L[\log n, 1]$ is the sorted version of the whole array X. Hence, *Simple_Mergesort* correctly sorts the input.

$\square$

Conclusion of the proof of Theorem 1

Proof of Theorem 1.

- By Lemma 2, applied to the root $(\log n, 1)$, the list $L[\log n, 1]$ is the sorted version of the whole array X. Hence, *Simple_Mergesort* correctly sorts the input.
- By Lemma 3, the work of each level is $O(n)$. Since the tree has $\log n$ levels, the total number of operations is $O(n \log n)$.

$\square$

Conclusion of the proof of Theorem 1

Proof of Theorem 1.

- By Lemma 2, applied to the root $(\log n, 1)$, the list $L[\log n, 1]$ is the sorted version of the whole array X. Hence, *Simple_Mergesort* correctly sorts the input.
- By Lemma 3, the work of each level is $O(n)$. Since the tree has $\log n$ levels, the total number of operations is $O(n \log n)$.
- Finally, all merges at the same level are performed in parallel, and each merge takes at most $O(\log n)$ time. Therefore, each level requires $O(\log n)$ time, and the overall running time is $O(\log^2 n)$.

$\square$

Complexity summary

Parallel time

Each level requires the merge seen in class:

$$T_{\text{level}}(n) = O(\log n).$$

Since there are $\log n$ levels:

$$T(n) = O(\log^2 n).$$

Total work

By Lemma 3, the work per level is linear:

$$W_{\text{level}}(n) = O(n).$$

Summing over all levels:

$$W(n) = O(n \log n).$$

Conclusion

Since $O(n \log n)$ is the optimal cost for comparison-based sorting, the algorithm is work **optimal**.

Complexity summary

Parallel time

Each level requires the merge seen in class:

$$T_{\text{level}}(n) = O(\log n).$$

Since there are $\log n$ levels:

$$T(n) = O(\log^2 n).$$

Total work

By Lemma 3, the work per level is linear:

$$W_{\text{level}}(n) = O(n).$$

Summing over all levels:

$$W(n) = O(n \log n).$$

Conclusion

Since $O(n \log n)$ is the optimal cost for comparison-based sorting, the algorithm is work **optimal**.

Question. What about the space complexity?

Complexity summary

Parallel time

Each level requires the merge seen in class:

$$T_{\text{level}}(n) = O(\log n).$$

Since there are $\log n$ levels:

$$T(n) = O(\log^2 n).$$

Total work

By Lemma 3, the work per level is linear:

$$W_{\text{level}}(n) = O(n).$$

Summing over all levels:

$$W(n) = O(n \log n).$$

Conclusion

Since $O(n \log n)$ is the optimal cost for comparison-based sorting, the algorithm is work **optimal**.

Question. What about the space complexity? $O(n \log n)$.

Complexity summary

Parallel time

Each level requires the merge seen in class:

$$T_{\text{level}}(n) = O(\log n).$$

Since there are $\log n$ levels:

$$T(n) = O(\log^2 n).$$

Total work

By Lemma 3, the work per level is linear:

$$W_{\text{level}}(n) = O(n).$$

Summing over all levels:

$$W(n) = O(n \log n).$$

Conclusion

Since $O(n \log n)$ is the optimal cost for comparison-based sorting, the algorithm is work **optimal**.

Question. What about the space complexity? $O(n \log n)$. Do you see why?

Space Complexity

Remark

Space can be reduced to $O(n)$ if we keep only the final list, instead of storing all lists associated with the nodes of the tree.

Corollary

A sequence of n elements can be sorted with

$$T(n) = O(\log^2 n), \qquad W(n) = O(n \log n)$$

on a CREW PRAM, using the *merge* seen in class.

Space Complexity

Remark

Space can be reduced to $O(n)$ if we keep only the final list, instead of storing all lists associated with the nodes of the tree.

Corollary

A sequence of n elements can be sorted with

$$T(n) = O(\log^2 n), \qquad W(n) = O(n \log n)$$

on a CREW PRAM, using the *merge* seen in class.

Remark The *Simple Mergesort* inherits the PRAM model of the *merge* used as a subroutine. Since the merge seen in class is formulated on a **CREW PRAM**, the sorting algorithm presented here is also CREW.

Can we do better?

Key point from the textbook

The textbook present in **Section 4.2** another *merge* procedure which has a better time complexity than the one shown at lesson.

Theorem 4

Let A and B be two sorted arrays, each of lenght n. Merging A and B can be done in $O(\log \log n)$ time and $O(n)$ work on a CREW PRAM model.

Proof is in the textbook but is too complex to be shown in this lecture.

Better Mergesort

Parallel time

Each level calls the merge procedure:

$$T_{\text{level}}(n) = O(\log \log n).$$

Since there are $\log n$ levels:

$$T(n) = O(\log n \log \log n).$$

Total work

The work per level is linear:

$$W_{\text{level}}(n) = O(n).$$

Summing over all levels:

$$W(n) = O(n \log n).$$

Conclusion

Since $O(n \log n)$ is the optimal cost for comparison-based sorting, the algorithm remains work **optimal** with a time complexity of $O(\log n \log \log n)$.

Better Mergesort

Parallel time

Each level calls the merge procedure:

$$T_{\text{level}}(n) = O(\log \log n).$$

Since there are $\log n$ levels:

$$T(n) = O(\log n \log \log n).$$

Total work

The work per level is linear:

$$W_{\text{level}}(n) = O(n).$$

Summing over all levels:

$$W(n) = O(n \log n).$$

Conclusion

Since $O(n \log n)$ is the optimal cost for comparison-based sorting, the algorithm remains work **optimal** with a time complexity of $O(\log n \log \log n)$.

Even with this approach, the algorithm still runs on a CREW PRAM model.

Pipelined MergeSort

Pipelining idea

- In **Simple MergeSort**, level h starts only after level $h-1$ is fully completed.
- Hence, computation proceeds **level by level**, with no overlap.
- In **Pipelined MergeSort**, partial information is forwarded upward as soon as it is available.
- Therefore, **different levels can be active simultaneously**.

Pipelined MergeSort

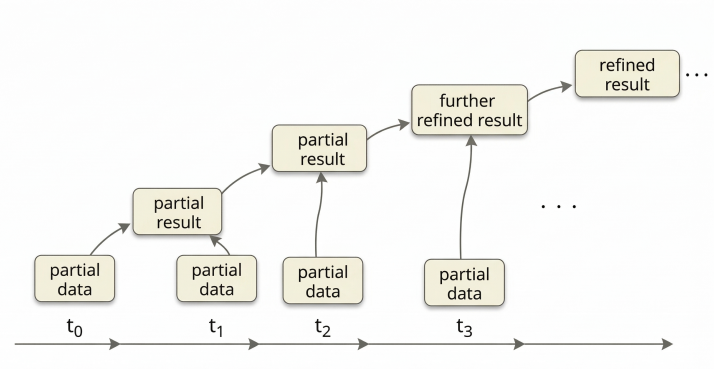
What changes in the next subsection

The textbook moves from a strictly level-by-level MergeSort to a pipelined version. To make this possible, we need a merge procedure that, after only a constant number of steps, already produces useful output for the next level. Only with this property can higher-level merges start in a *true pipelined fashion*, instead of waiting too long before receiving usable information.

Important remark. This improvement does *not* follow automatically from the merge procedures seen so far. With that merge, an initial $O(\log n)$ time is spent before useful output becomes available, so the next merge cannot start in true pipelined fashion.

Main intuition. In the pipelined version, partial information is propagated upward and improves higher-level nodes stage by stage.

Pipelining intuition



Interpretation. In the pipelined version, higher nodes are refined progressively as partial information becomes available from below.

Complexity of Pipelined Mergesort

Theorem 5

Using the Pipelined Mergesort, sorting n elements can be done in $O(\log n)$ time, using a total of $O(n \log n)$ operations.

Concurrent read is used in the procedures involving this algorithm. Hence, this algorithm runs on a CREW PRAM model. This algorithm can be modified to run on the EREW PRAM, but the details required are nontrivial.

For more details see the **Section 4.3.2** of the textbook.

Lower bound for sorting

Comparison lower bound

It is well known that sorting n elements requires $\Omega(n \log n)$ comparisons in the binary decision-tree model.

Remark: In comparison PRAM with $p \leq n$, if at most n processors are available, then in one parallel step we can perform at most $O(n)$ comparisons.

Time lower bound

Therefore, the parallel time cannot be smaller than $\Omega\left(\frac{n \log n}{n}\right) = \Omega(\log n)$.

Corollary

Any comparison-based sorting algorithm running in $O(\log n)$ time on a PRAM with $p \leq n$ processors is **time-optimal**.

Comparison of the three MergeSort variants

Algorithm	**PRAM model**	**Time complexity**	**Work complexity**
Simple Mergesort	CREW	$O(\log^2 n)$	$O(n \log n)$ ✓
Better Mergesort	CREW	$O(\log n \ \log\log n)$	$O(n \log n)$ ✓
Pipelined Mergesort	CREW*	$O(\log n)$ ✓	$O(n \log n)$ ✓

Note: It can be further refined to be EREW, but the details are nontrivial and are not covered in this lecture.

Remark: *Pipelined Mergesort* is both time-optimal and work-optimal.

Thanks

Thanks